

QUANTIZATION BENCHMARKING

Six Precision Formats, One Question: What Do You Trade Away?

A full breakdown of FP16, FP8, and four INT4/INT8 formats (AWQ, GPTQ, bitsandbytes) across Qwen 2.5 and Llama 3 model families — accuracy, speed, GPU utilization, tool-call quality, and energy, from 270 agent trials.

NVIDIA RTX 5090 (32GB) · vLLM / Transformers · pynvml telemetry @ 50ms · 19 slides

The results in six numbers

Full detail follows — this is the headline picture before the deep dive.

270

AGENT TRIALS RUN

6 formats x 6 models x 3 agents x 10 trials (minus n/a combos)

100%

QWEN 2.5 SUCCESS

at FP16, FP8, and INT4-AWQ alike, at every size 3B-14B

-49.5%

PEAK ENERGY SAVINGS

INT4-AWQ vs FP16 on Qwen2.5-14B, at equal accuracy

-66%

PEAK FOOTPRINT CUT

INT4-AWQ weight size vs FP16 on Qwen2.5-14B

7%

WORST-CASE SUCCESS

Llama-3.2-1B under INT4 GPTQ, down from a 45% baseline

2.3x

ENERGY GAP, BEST VS WORST FORMAT

bitsandbytes-INT8 vs AWQ, same Llama models

Six precision formats, tested head-to-head

Same agents, same tasks, same GPU — only the weight format changes between runs.

1

Qwen 2.5 Formats

FP16 (baseline)
FP8 static per-tensor
INT4 - AWQ

3B, 7B, 14B sizes
9 model x format combos

2

Llama 3 / 3.2 Formats

FP16 (baseline)
INT4 - AWQ, GPTQ
INT8 - GPTQ, bitsandbytes
INT4 - bitsandbytes

1B, 3B, 8B sizes
18 model x format combos

3

Agents & Trials

Support Agent
IT Helpdesk Engineer
Financial Analyst

10 trials per model
per format per agent

4

Hardware & Stack

NVIDIA RTX 5090
32GB VRAM

vLLM (FP16, AWQ, GPTQ,
bitsandbytes)
HF Transformers (FP8)

How each metric is defined

Precise definitions, straight from the benchmark harness.

Success Rate

A trial passes only if every tool in the task's `must_call` set executes without error AND no tool in `must_not_call` fires. Binary per trial, averaged over 10.

Achieved TFLOPS

$(2 \times \text{generated_tokens} \times \text{params_in_billions} \times 1e9) / (1e12 \times \text{duration_sec})$ - a compute-throughput heuristic, not a hardware ceiling.

Total Energy (J)

Trapezoidal integration of instantaneous power draw over the generation window - what the GPU actually spent, not power x time at a single reading.

Tokens / Sec

$\text{generated_tokens} / \text{generation_duration_sec}$, measured only during the model's active generation window.

Avg GPU Util (%)

Mean of pynvml utilization samples (50ms interval) across the generation window - the share of time the SM was actively issuing work.

Parse Errors / Correct Order

Regex-parsed Action blocks that failed JSON validation, and whether the executed tool sequence matched the task's expected order exactly.

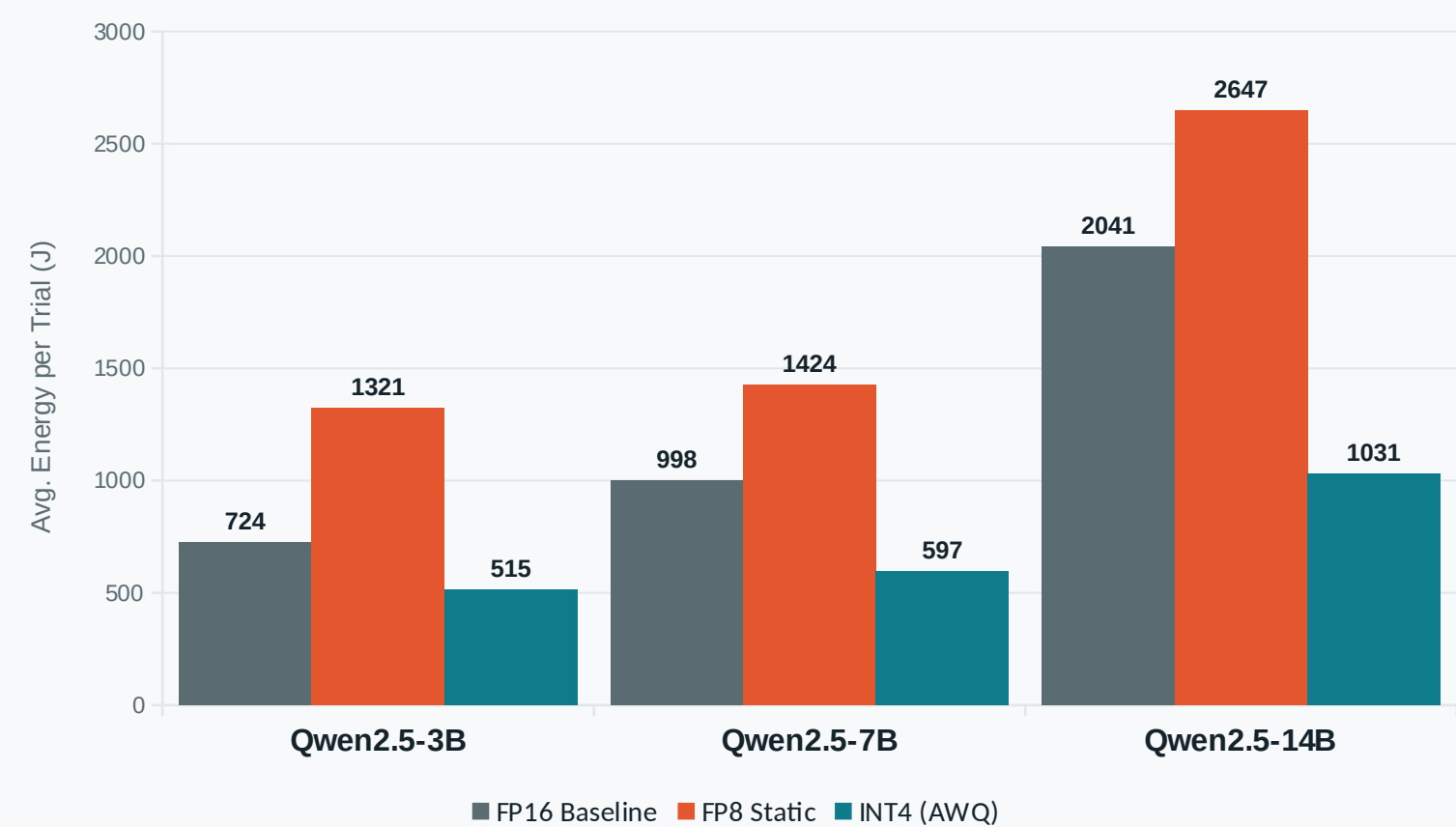
Qwen 2.5 — full results, all 9 combinations

Model	Format	Success	Tok/s	TFLOPS	Weights	VRAM	Power	Energy
Qwen2.5-3B	FP16	100%	76.8	0.46	5.75 GB	7.64 GB	268 W	724 J
Qwen2.5-3B	FP8	100%	30.2	0.18	3.16 GB	8.00 GB	178 W	1321 J
Qwen2.5-3B	INT4-AWQ	100%	61.9	0.37	1.92 GB	3.82 GB	162 W	515 J
Qwen2.5-7B	FP16	100%	78.0	1.09	14.19 GB	16.27 GB	456 W	998 J
Qwen2.5-7B	FP8	100%	32.8	0.46	8.11 GB	17.10 GB	264 W	1424 J
Qwen2.5-7B	INT4-AWQ	100%	72.5	1.01	5.19 GB	7.22 GB	234 W	597 J
Qwen2.5-14B	FP16	100%	42.0	1.18	27.51 GB	29.59 GB	494 W	2041 J
Qwen2.5-14B	FP8	100%	19.1	0.54	15.21 GB	30.95 GB	293 W	2647 J
Qwen2.5-14B	INT4-AWQ	100%	42.9	1.20	9.29 GB	11.35 GB	258 W	1031 J

Every cell hits 100% success — differences are purely in speed, footprint, and energy.
Quantization & Energy Benchmarking of Agentic LLMs

INT4 quantization nearly halves energy use — with zero accuracy cost

Every Qwen 2.5 size (3B / 7B / 14B) hit 100% task success at FP16, FP8, and INT4-AWQ alike.



ENERGY SAVED BY INT4 vs FP16

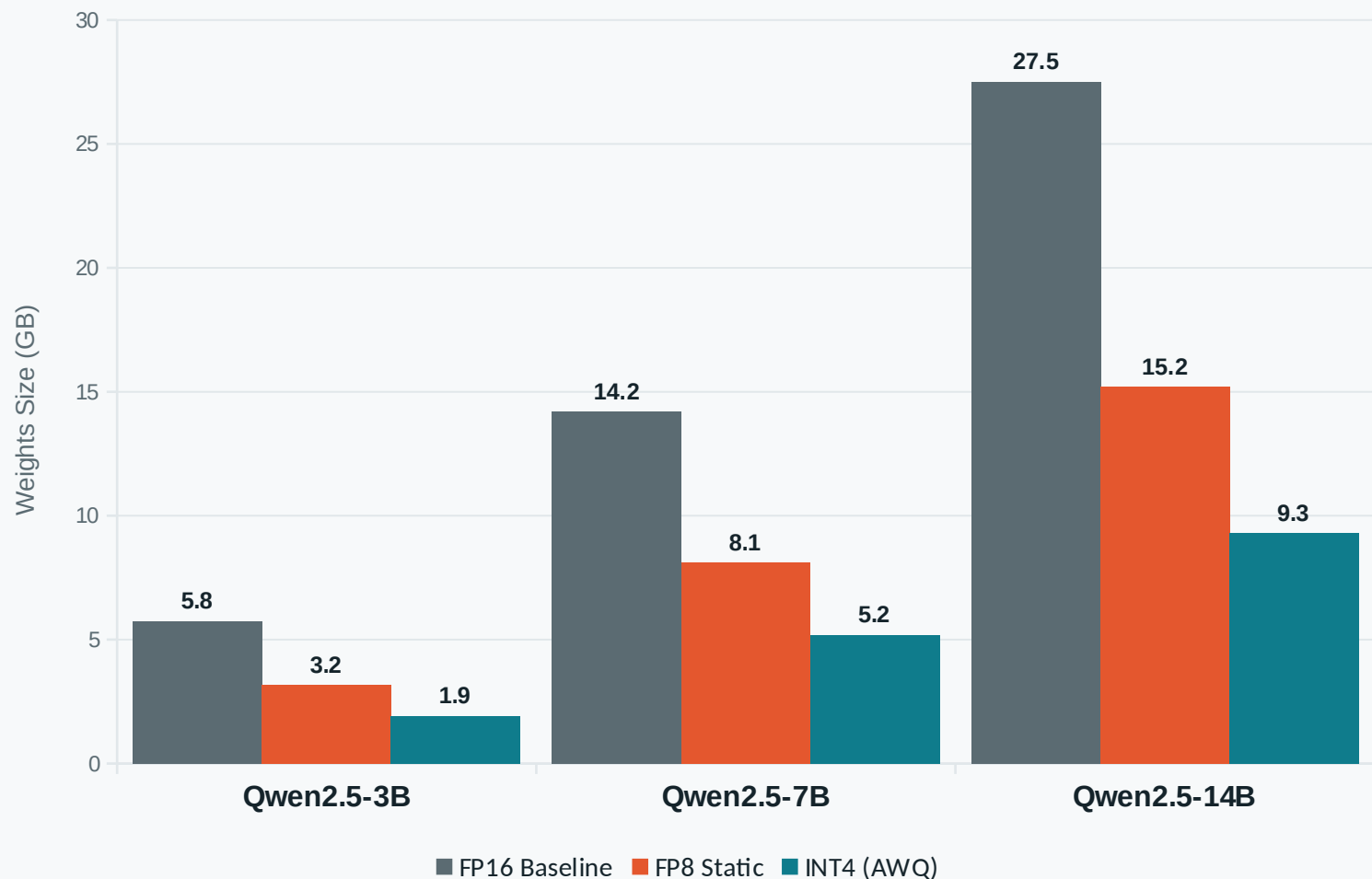
-49.5%
14B model

-40.2%
7B model

-28.8%
3B model

Average across all three sizes: -39.5% energy at equal accuracy.

...while the model footprint shrinks by up to 66%



READ THE CHART

INT4-AWQ consistently compresses weights to roughly a quarter of FP16 size - 27.5GB to 9.3GB on the 14B model.

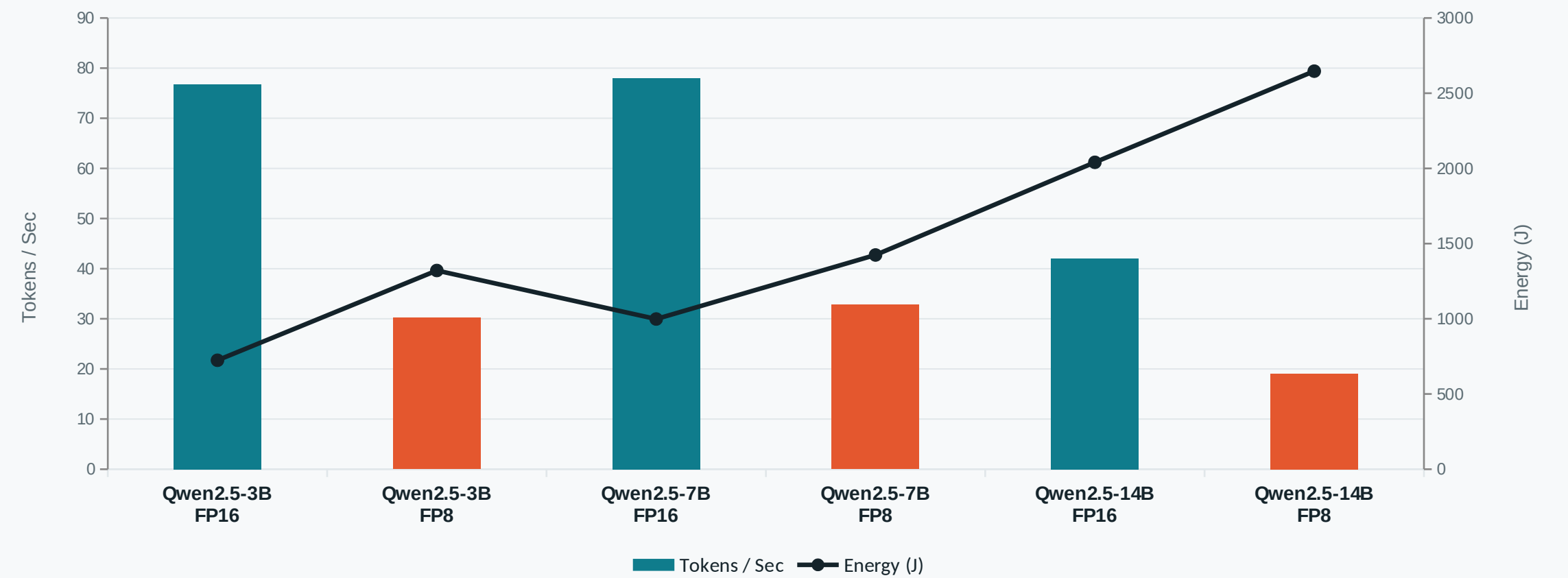
FP8 only reaches about half the FP16 size, less compression than INT4 for a format that also runs slower (next slide).

Practical effect: a 14B model in INT4 fits comfortably alongside other workloads on a single mid-range GPU.

FINDING 03 - A SURPRISE

FP8 is the odd one out — slower and less efficient than FP16

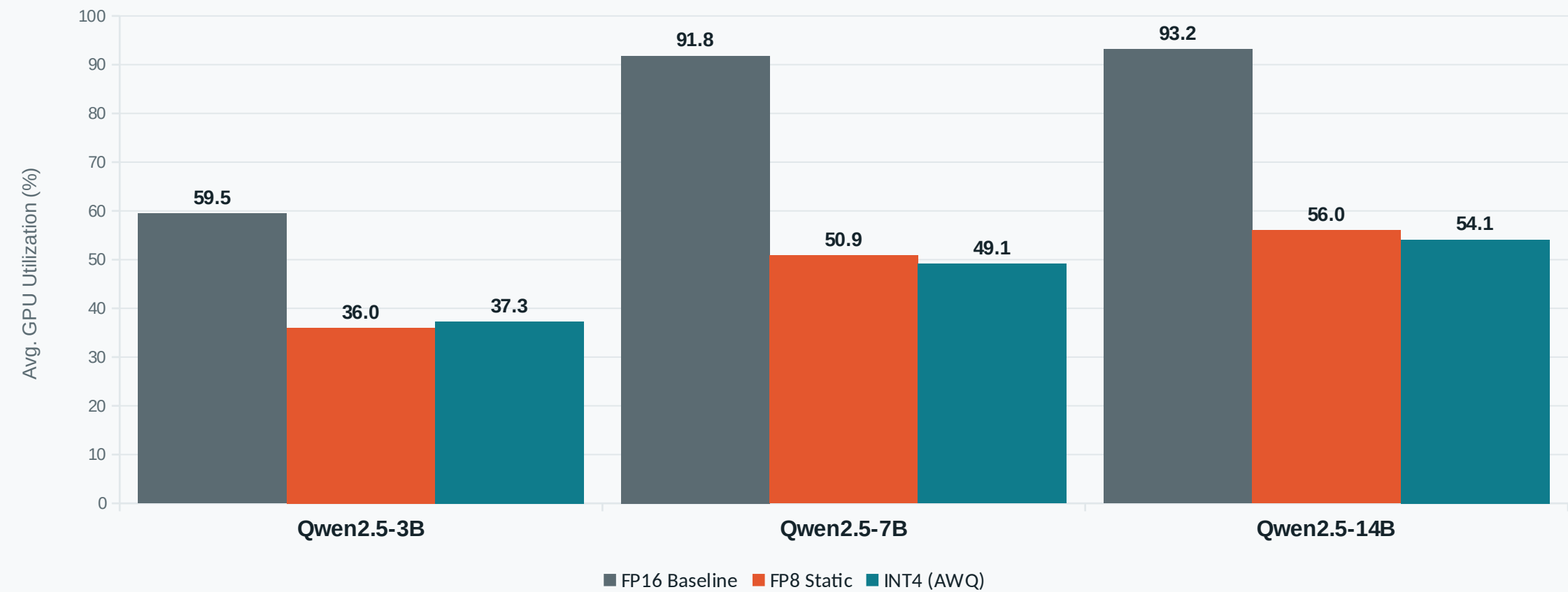
Bars = tokens/sec (left) · Line = energy per trial in Joules (right)



FP8 static per-tensor loses 55-61% throughput and consumes up to 82% more energy than FP16 - explained on the next slide by GPU utilization data.

GPU utilization shows exactly where FP8 loses its speed

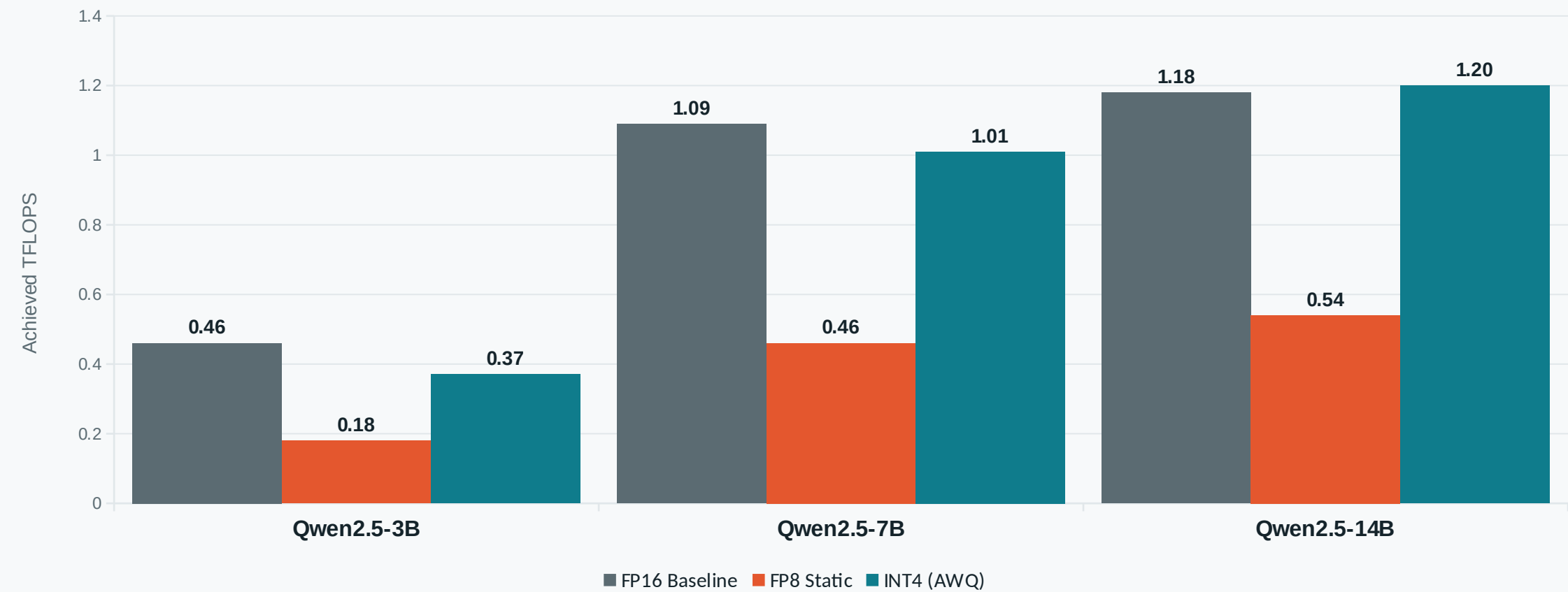
Average % of time the GPU's compute units were actively issuing work during generation.



Both FP8 and INT4 leave the GPU under-utilized versus FP16 - but INT4 still wins on speed because it moves 4x less data per token than FP8's 2x, more than offsetting the lower utilization.

INT4 is the only format that matches FP16's compute efficiency

Achieved TFLOPS = (2 x tokens generated x parameters) / duration - higher means more useful compute per second.



On the 14B model, INT4-AWQ actually edges out FP16 in achieved TFLOPS (1.20 vs 1.18) - quantized weights let the GPU push more useful tokens/sec despite the lower-precision math.

Llama — full results, all 18 combinations

Llama-3.2-1B

Format	Succ	Tok/s	E (J)
FP16	45%	168	929
AWQ	27%	132	560
BNB-4bit	30%	123	1095
BNB-8bit	23%	51	1461
GPTQ-4bit	7%	136	752
GPTQ-8bit	37%	136	809

Llama-3.2-3B

Format	Succ	Tok/s	E (J)
FP16	100%	101	865
AWQ	93%	78	608
BNB-4bit	97%	72	688
BNB-8bit	100%	30	1277
GPTQ-4bit	97%	80	595
GPTQ-8bit	100%	81	735

Llama-3-8B

Format	Succ	Tok/s	E (J)
FP16	100%	75	1262
AWQ	97%	65	628
BNB-4bit	77%	52	883
BNB-8bit	93%	23	1377
GPTQ-4bit	93%	67	625
GPTQ-8bit	97%	66	777

Succ = Success Rate (10 trials) · Tok/s and Energy averaged across 3 agents.

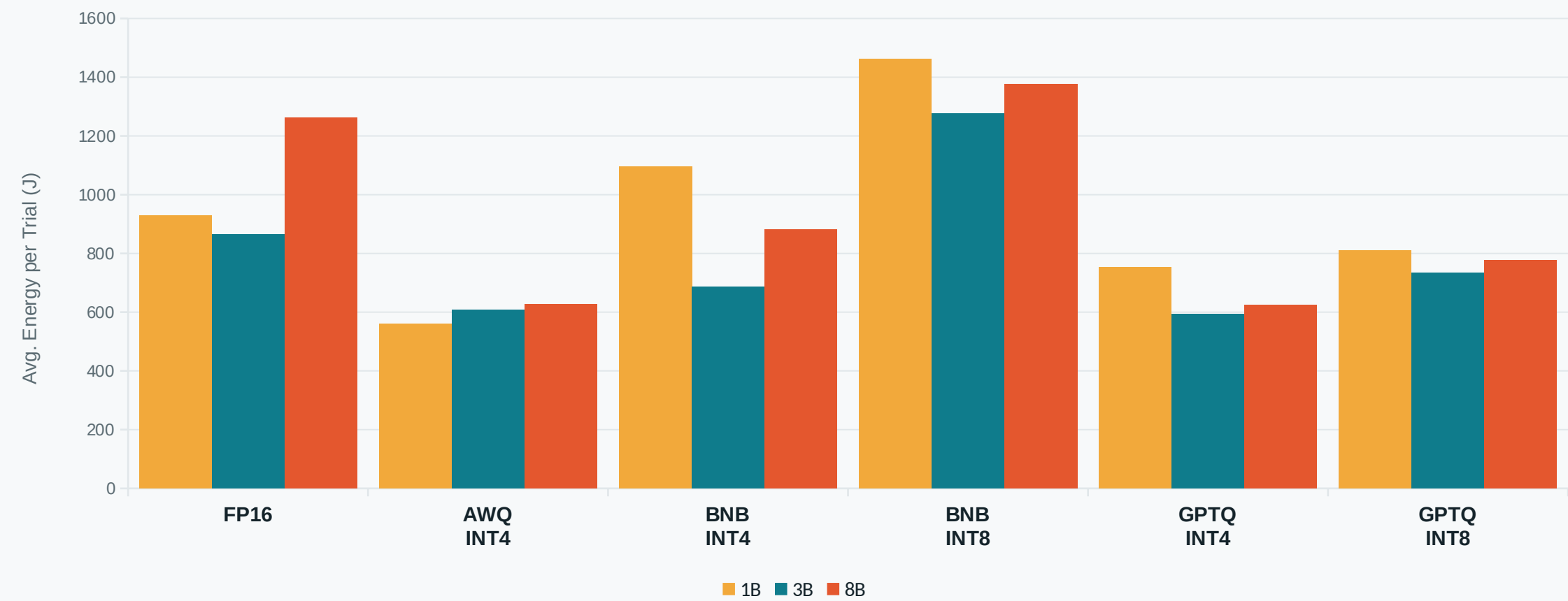
Llama's accuracy is far more sensitive to quantization format

Format	1B	3B	8B
FP16 (baseline)	45%	100%	100%
INT4 - AWQ	27%	93%	97%
INT4 - bitsandbytes	30%	97%	77%
INT8 - bitsandbytes	23%	100%	93%
INT4 - GPTQ	7%	97%	93%
INT8 - GPTQ	37%	100%	97%

Notice: 3B and 8B stay largely intact under every format (mostly 77-100%), but the already-fragile 1B model degrades further under quantization - dropping as low as 7% with INT4 GPTQ, worse than its already-weak 45% FP16 baseline.

Energy cost per format follows model size, with one exception

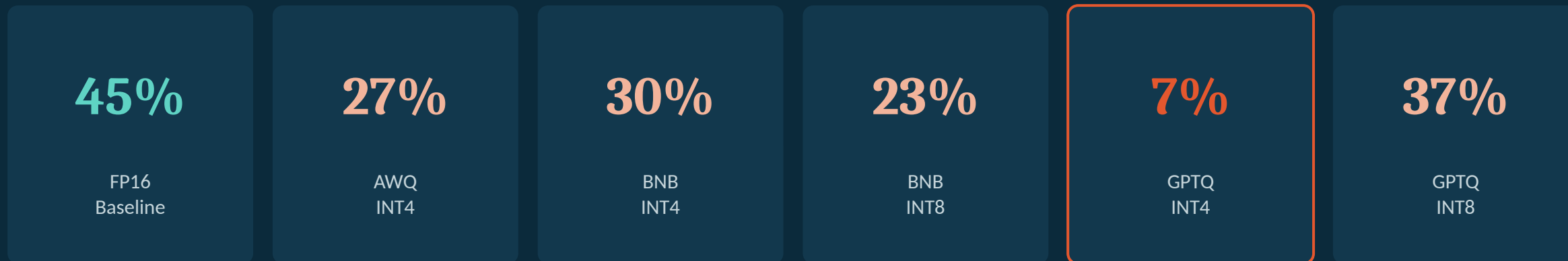
Average energy per trial, Joules — one series per model size, six quantization formats.



bitsandbytes-8bit (BNB INT8) is the exception - the most expensive format at every size, even beating out the unquantized 8B baseline.

Quantization hits hardest where a model is already weak

Llama-3.2-1B — Success Rate by Format

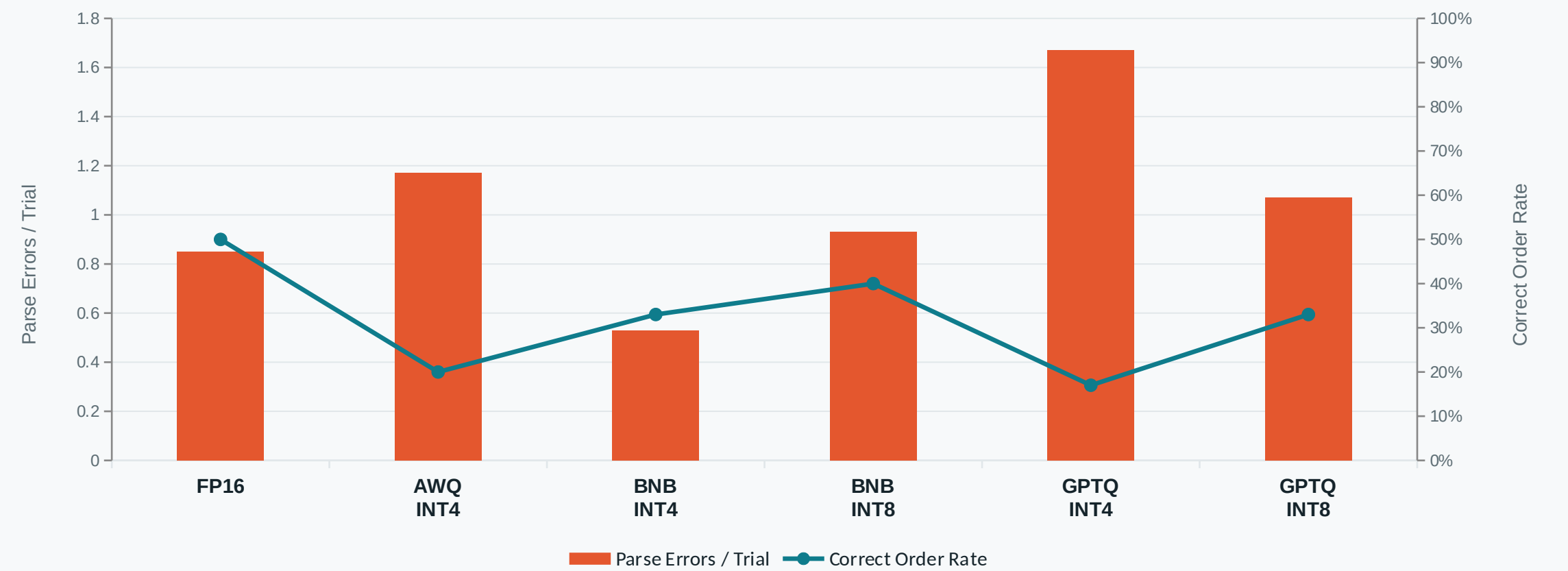


Why this happens: at 1B parameters the model already sits close to a capability floor for structured tool-calling - its margin for the small numerical errors quantization introduces is thin. GPTQ's calibration approach appears especially disruptive at this scale, while the 3B and 8B Llama models have enough headroom to absorb the same quantization with only single-digit accuracy shifts.

Takeaway: quantization risk should be evaluated per model size, not assumed to be uniform across a family.

The collapse traces to formatting, not reasoning

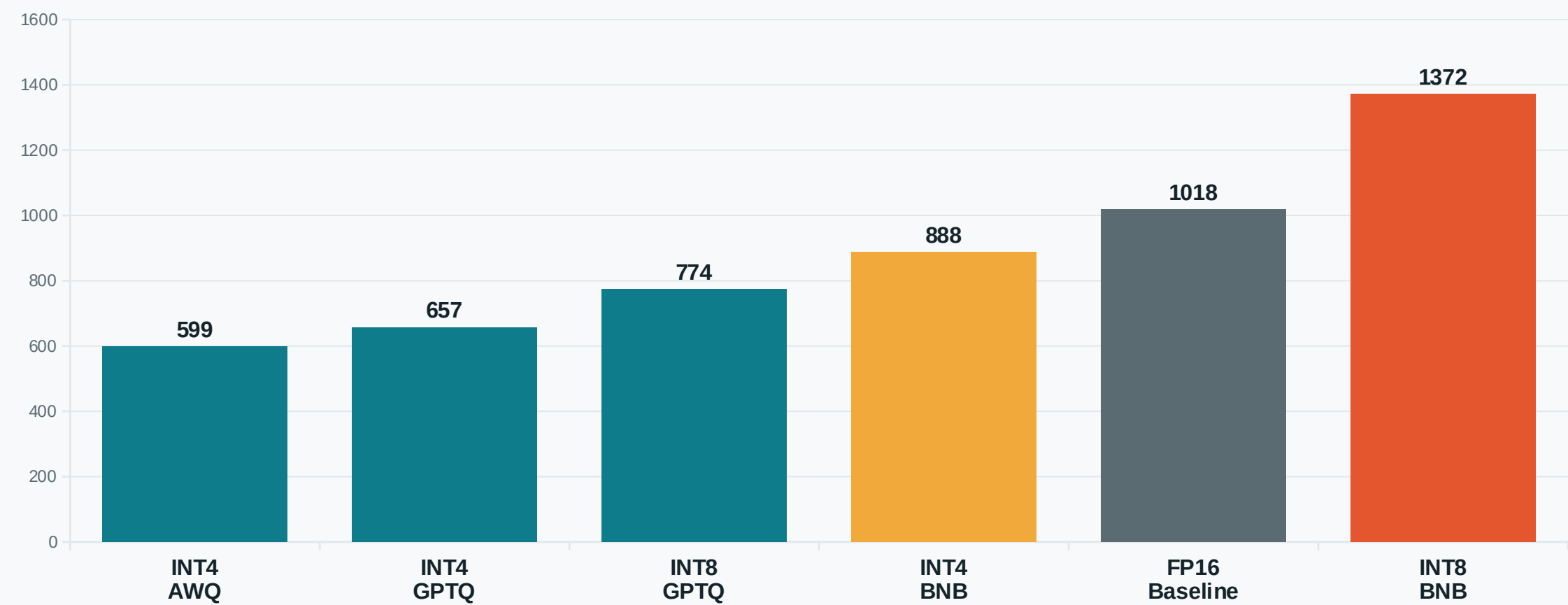
Llama-3.2-1B: parse errors per trial (bars) vs. correct tool-call order rate (line) by format.



GPTQ-4bit has both the most parse errors (1.67/trial) and the lowest correct-order rate (17%) - the same format that produced the 7% success floor. Quantization noise is corrupting the model's JSON formatting before it ever reaches the reasoning.

AWQ is the most energy-efficient format — bitsandbytes-8bit the least

Average energy per trial across Llama-3.2-1B, 3B, and 8B, one bar per quantization format.



bitsandbytes-8bit is 2.3x more energy-hungry than AWQ despite being a "lighter" 8-bit format - its dequantization overhead dominates at inference time and it even trails the FP16 baseline.

Which format should you actually deploy?

Format	VRAM ↓	Speed	Accuracy Risk	Verdict
INT4 - AWQ	~65-75%	High	Low (models ≥3B)	Best all-round choice
INT4 - GPTQ	~65-75%	High	Medium - risky <3B	Strong, avoid on tiny models
INT8 - GPTQ	~35-45%	High	Low	Safe, moderate savings
INT4 - bitsandbytes	~65-75%	Medium	Medium	OK if AWQ unavailable
FP8 (this stack)	~45-50%	Low	Low	Skip until vLLM-native
INT8 - bitsandbytes	~35-45%	Low	Medium	Avoid - slow & costly

Key takeaways

01

INT4-AWQ is close to a free lunch

Qwen 2.5 (3B-14B) kept 100% success while cutting energy up to 49.5% and VRAM up to 66%.

02

Not all formats are equal

AWQ and GPTQ-4bit beat bitsandbytes on speed and energy across both model families.

03

FP8 underperformed here

Slower and pricier than FP16 in this stack - a software-path artifact (Transformers vs vLLM), not a hardware limit.

04

Small models are the risk zone

Llama-3.2-1B degraded under every format tested, cratering to 7% with GPTQ-4bit.

05

Failures are formatting, not reasoning

Parse errors and broken tool-call order - not bad decisions - drive the 1B model's collapse.

06

"Lighter" isn't always cheaper

bitsandbytes-8bit used more energy per trial than the FP16 baseline it was meant to shrink.

Where this study should go next

The 270-trial sweep answers the headline question — quantization can be nearly free for mid-size models — but raises new ones.

1

More trials on volatile models

1B-scale success rates swing sharply on $n=10$; widen to 30+ trials to get real confidence intervals.

2

Run FP8 on vLLM natively

Confirm whether FP8's slowdown is a Transformers backend artifact or a genuine precision cost.

3

Add a mid-size Llama gap

Test a ~2B Llama checkpoint to see where the accuracy cliff between 1B and 3B actually sits.

4

Per-task error analysis

Pull trace-level data for quantized runs to see whether failures are formatting errors or reasoning errors.

Repository: github.com/Naresh-x86/Quantization-Benchmarking